

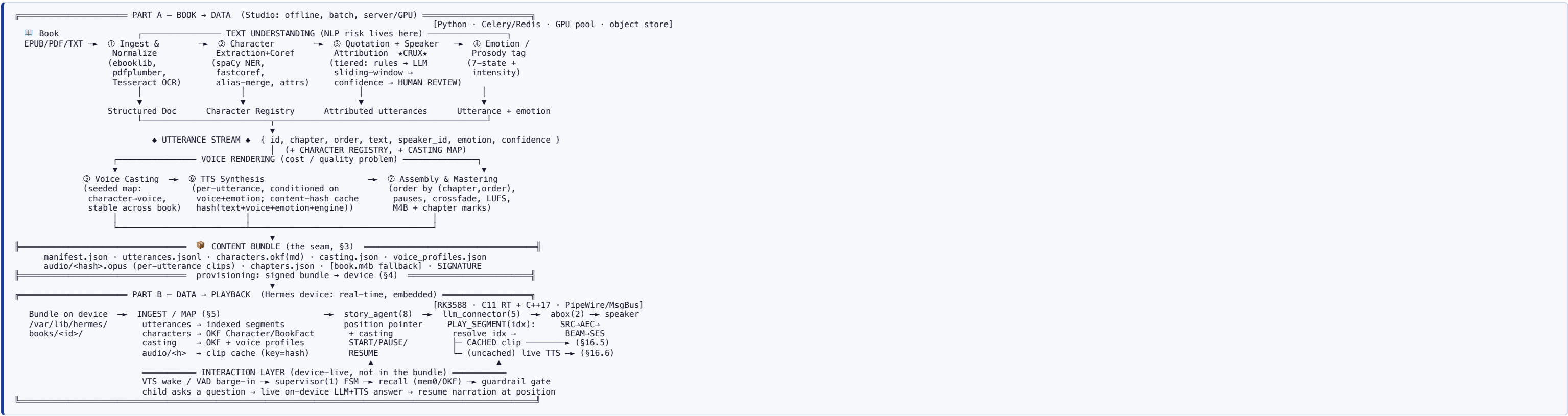
Hermes — End-to-End Pipeline: Book → Data → Playback

Scope: the complete path from a raw book file to interactive multi-voice playback on the Hermes device, across the two build paths that make up the product. **Document status:** Draft v0.3 · 2026-07-12 · *for review* · **uncommitted Relationship to other docs:** this is the detailed design behind ARCHITECTURE.md **UC-7** and **\$17.7 "Book pipeline (offline, batch)"**. It also specifies the **bundle contract** that Part I §16.5 (cached-clip playback) consumes, and the role-load inputs docs/memory_architecture.md §4.1 assembles for interactive turns. Source design for the offline half: Book-to-Voice Architecture.pdf (Multi-Voice Audiobook Technical Design).

Rev	Date	Changes
v0.1	2026-07-07	Initial end-to-end design: Part A stages, bundle contract, Part B mapping, transport, milestones.
v0.2	2026-07-12	Studio M0–M1 implemented (§2.1): rules tiers + scanned-PDF/OCR ingest, validated end-to-end on a real scanned book (§2.2). Added Stage ③ character dossiers (§2.3), UC-8 character-conversation design (§6.1), DR-1 TTS placement decision (§6.2). §9/§11 statuses updated.
v0.3	2026-07-12	Stage ③ implemented (contract v0.2): scenes.json (Layer 3, spoiler gates), utterance addressee_id / scene_id, dossier-enriched characters.okf cards — rules tier + c laude -CLI LLM tier (cached, deterministic). Validated on the Aladdin scan; M2.5 Studio half done (§2.3, §9).

One product, two build paths, one contract. A **Book-to-Voice Studio** (offline, server/GPU, Python) turns a book into a **content bundle**; the **Hermes device** (real-time, embedded, C/C++ on RK3588) plays that bundle and adds live interaction. They never share a codebase — they meet only at the **bundle contract** (§3). Everything upstream of the contract is batch and cost-bound; everything downstream is real-time and latency-bound.

1. The whole pipeline at a glance



Reading it: Part A is run **once per book**, offline, to produce a **bundle**. Part B loads that bundle and plays it **in real time**, layering interaction (wake word, barge-in Q&A, memory, guardrails) that the bundle does not contain. The two halves are independently buildable and testable because the **bundle contract (§3)** is the only thing they share.

2. PART A — Book → Data (the Studio, offline)

Server-side batch pipeline. Not on the device; never cross-compiled for aarch64. Full stage detail in Book-to-Voice Architecture.pdf ; condensed here with the **artifact each stage emits**.

#	Stage	Does	Emits	Key tools / method
①	Ingestion & normalization	any format → clean structured text + metadata; strip headers/footers, de-hyphenate, unify quotes, detect chapters	Structured Doc + ToC	ebooklib (EPUB); as-built PDF tier (§2.2) : poppler <code>pdftotext</code> (text layer) / <code>pdftoppm</code> +Tesseract TSV (scanned); heading regex
②	Character extraction & coreference	find every character, unify aliases, mine attributes for casting	Character Registry {id, canonical_name, aliases[], gender, age_hint, personality, dialect}	spaCy NER (<code>en_core_web_trf</code>), fastcoref, alias-merge (rules + LLM adjudication), LLM attribute mining
③	Quotation detection & speaker attribution ★	split quoted speech vs narration; attribute each quote to a character or NARRATOR	speaker_id on every span + confidence	Tiered, cheapest-first : explicit "said X" → anaphoric → alternating-turn → LLM sliding-window (2–3k tok, +registry) → confidence gate $\tau \approx 0.75$ → human-review UI
④	Emotion / prosody tagging	attach tone + intensity per utterance	emotion label + intensity 0–1	cue extraction ("whispered/snapped") + LLM/fine-tuned classifier → 7-state : neutral, joy, sadness, anger, fear, surprise, tenderness
⑤	Voice casting	deterministic, seeded character→voice map, stable across all chapters, unique per major character	Casting Map character_id → voice_profile_id + Voice Profiles {engine, seed, ref_audio, pitch, rate, style}	fixed voice library (assignment/Hungarian) or seeded voice-clone; narrator = distinct neutral voice
⑥	TTS synthesis	render one audio segment per utterance, conditioned on voice+emotion	per-utterance audio clips	content-hash cache hash(text+voice+emotion+engine_version) → only misses hit the engine; cloud (ElevenLabs/Cartesia/PlayHT) to prototype, self-hosted GPU (XTTS-v2/StyleTTS2/Fish/Kokoro) for the hot path
⑦	Assembly & mastering	order, space, master	M4B (+ per-utterance clips kept for Hermes)	order by (<code>chapter, order</code>) ; pauses (sentence ≈250 ms, paragraph ≈600 ms, scene ≈1.2 s); LUFS −18...−23; ffmpeg + chapter marks

★ §3 is the crux — ~80% of perceived quality and engineering risk. Human review here is **part of the product, not a fallback**: low-confidence attributions are fixed cheaply *before* any audio is rendered.

Operational shape (Studio): async job queue (Celery+Redis), stateless per-stage workers (NLP on CPU, TTS on a GPU pool), **every stage checkpointed to object storage** (resume mid-book), content-hash caching at utterance granularity (edits re-render only what changed), a metadata DB + review web app. This is a cloud/server system, not device firmware.

2.1 Implementation status — `studio/` (as of 2026-07-12)

The Studio is **no longer design-only**. The rules tiers of stages ①–⑤+⑦(data) are implemented in `studio/` (Python, stdlib-only for the default tiers) and validated end-to-end.

Stage	Rules tier (default)	Status	Optional/ML tier	Status
① Ingest	.txt/.md prose · PDF: text-layer (<code>pdftotext</code>) + scanned/OCR (<code>pdftoppm</code> + <code>tesseract</code>)	✔ built & tested	EPUB (<code>ebooklib</code>) ✔ · vision-LLM page extractor	🛑
② Registry	capitalized-name freq + pronoun gender + place filter + gendered-title names	✔	spaCy NER (<code>--nlp spacy</code>)	⚠ wired, unvalidated
③ Attribution ★	5-tier cascade (explicit 0.97 → pronoun-gender → nearest-name → alternating → UNKNOWN) + τ review queue	✔	LLM sliding window (<code>--attributor llm</code>)	🛑 extension point
④ Emotion	cue-word → 7-state, intensity cap 0.6	✔	classifier / LLM pass	🛑 extension point
⑤ Casting	SHA-256-seeded, per-gender library, prominence-ordered — zero RNG	✔	voice-clone library	🛑
⑥ TTS synthesis	—	🛑 next milestone (DR-1, §6.2)		
⑦ Bundle/data	full §3 contract emit + validator; idempotent re-runs	✔	mastering/M4B	🛑
⑧ Knowledge (scenes + dossiers)	scene segmentation (location-cue + chapter), addressee post-pass (vocative + two-party), evidence-only dossiers	✔ built & tested	persona authoring via <code>claude CLI (--knowledge llm)</code>, content-hash cached ⇒ deterministic rebuilds	✔ validated on sample book

Verified invariants (CI-able, `studio/tests/` — 22 tests): contract-valid bundles; byte-identical rebuilds (fresh-vs-fresh `diff -r` clean); UTF-8-pinned subprocess decoding (locale-independent); atomic, crash-safe OCR cache writes; 300 s timeouts on all external tools.

2.2 Stage ① scanned-PDF ingest — as-built design & validation

Validated against a worst-case real input: a **2004 Ladybird/Disney "Read it yourself" Aladdin** — 16 PDF pages, every page a full-page **image scan** (zero text layer), each a two-page spread with text blocks scattered around illustrations, plus cover/copyright/series-blurb pages.

Pipeline (`studio/ingest/pdf.py` , zero Python deps — poppler + tesseract CLIs): 1. **Tier probe** — `pdftotext` ; ≥ 200 chars $\Rightarrow$ text-layer path, else OCR path. 2. **Geometry** — `pdfinfo` page boxes; landscape ($w > h \times 1.05$) $\Rightarrow$ spread, split at the gutter into printed pages L/R (whole-spread OCR interleaves the two columns — measured failure). 3. **Per-page render+OCR, cached** — `pdftoppm` 300 DPI $\rightarrow$ `tesseract` TSV; artifacts cached under `~/.cache/hermes-studio/ocr/<pdf-sha12>--<dpi>--<tess-ver>--<poppler-ver>/pNNN{L,R,F}.{png,tsv}` , written atomically. Each printed page is an **independent, idempotent unit of work** — the §2 "stateless per-stage workers" shape at page granularity; a batch runner can fan out without code changes, and re-runs are pure cache hits. 4. **Word-confidence gate** — keep conf ≥ 75 words containing a letter. Measured separation on the sample: real text 86–97, illustration garbage 0–72. This is what makes picture books ingestible at all. 5. **Cleanup & classification** — known OCR fixups (“T wish $\rightarrow$ “I wish), page-number/symbol line drops; each printed page classified `body | matter | empty` (publisher/legal/series keyword regex) — covers, copyright, "About this book," back cover are auto-dropped. 6. **Assembly** — body blocks $\rightarrow$ paragraphs in spread order; blocks with unbalanced quotes or no terminal punctuation merge with the next (capped 1200 chars), repairing large-print splits.

Validation result (M0 exit evidence): contract-VALID bundle; 28 utterances = faithful transcription (25 narration / 3 dialogue, **all 3 explicit-tier @ 0.97**, review queue empty); registry exactly {Aladdin ♂, Jafar ♂, Jasmine ♀, Sultan ♂} — "Agrabah" correctly rejected as a place; deterministic rebuild verified. Known rules-tier limits (by design, ML tiers refine): lowercase common-noun characters ("the genie") are not registered; single-gender pronoun voting can mis-attribute across an antecedent boundary.

2.3 Stage ⑧ — Story knowledge: scenes + character dossiers (built, v0.2)

Why: the utterance stream is complete for *playback* but nearly empty for *interaction* (UC-8). Interaction needs three data layers; Stage ⑧ (studio/knowledge/) produces layers 2–3:

Layer	Content	Bundle artifact	Status
L1	what was said, by whom, in order	utterances.jsonl	✔️ (v0.1)
L2	who each character is — persona, relationships, style	characters.okf/ dossier cards	✔️
L3	what happened , where, in story order	scenes.json	✔️

Layer 3 — scenes. Rules tier segments the utterance stream at chapter boundaries and narration lines that move the action ("She ran off *into the market*" — location-cue lexicon). Each scene gets location, participants (speakers + mentioned registry characters), a verbatim first-narration-sentence summary, and max_order = order_end — the **spoiler gate** role-load filters against the listener's position. Every utterance is stamped with its scene_id; dialogue gains a rules-tier addressee_id (quote-initial/final vocative, else two-party-scene inference; never guessed — stays null when unproven, e.g. lines spoken to an unregistered character).

Layer 2 — dossiers. Two tiers, same philosophy as attribution: - **Rules tier (always runs — the evidence base):** verbatim quotes, style statistics, relationships the text proves (speaker↔addressee exchanges > scene co-presence), emotion range from Stage-④ tags, and witnessed-scene knowledge items each carrying that scene's max_order. Persona fields stay **empty** — the rules tier never invents personality. - **LLM tier (--knowledge llm):** authors role, traits, backstory, speaking-style direction, catchphrases, relationship phrasing, and refined per-scene knowledge facts via the claude CLI, constrained to the book text (+ public canon for famous works, subordinated to text). Responses are **content-hash cached** (sha(book+char+prompt_version)), so rebuilds are byte-identical after the first run. Every card ships needs_review: true — persona is product content and passes the same human gate as attribution.

Validated on the Aladdin scan: 4 scenes (city 0–4 · market 5–7 · cave 8–15 · sea 16–27), LLM dossiers with correct roles/relationships for all 4 cast characters (including "Genie — his friend, whom he sets free," a character the rules registry cannot see), all knowledge items spoiler-gated, contract v0.2 VALID, both tiers deterministic.

3. The Content Bundle — the contract (the seam)

The **only** interface between the two halves. A per-book, versioned, **signed** directory. Hermes consumes exactly this and nothing about how it was produced.

```
books/<book_id>/
manifest.json      # book_id, title, author, engine_version, contract_version,
                   # chapter list, per-file integrity hashes, signature ref
utterances.jsonl   # ONE row per spoken/narrated span (the utterance stream):
                   # { id:uuid, chapter:int, order:int, text:str,
                   #   speaker_id:str|"NARRATOR", emotion:{label,intensity}, confidence:float,
                   #   audio:"audio/<hash>.opus", source_span:[start,end],
                   #   addressee_id:str|null, scene_id:str|null } [v0.2]
scenes.json        # [v0.2] Layer-3 story knowledge: ordered scenes over utterance ranges
                   # { scene_id, chapter, order_start, order_end, location,
                   #   participants[char_id], summary, max_order }
                   # max_order == order_end is the SPOILER GATE: role-load reveals a
                   #   scene's facts only when the listener's position has passed it
characters.okf/     # OKF markdown: one Character doc per character — the UC-8 role-load card
                   # [v0.2] frontmatter + dossier: role, traits, emotion_range,
                   #   generated_by (rules|llm), needs_review; body: backstory, speaking
                   #   style, catchphrases, relationships, verbatim quotes, and
                   #   spoiler-gated knowledge items (each tagged max_order)
casting.json        # character_id -> voice_profile_id (seeded, stable across chapters)
voice_profiles.json # voice_profile_id -> { engine, seed, pitch, rate, style, ref_audio? }
chapters.json      # chapter index -> title, first/last order, pause profile
audio/             # per-utterance clips, filename = hash(text+voice+emotion+engine_version)
<hash>.opus        # -> this hash IS Hermes's clip-cache key (no re-derivation on device)
book.m4b           # OPTIONAL linear fallback (non-interactive players); Hermes ignores it
SIGNATURE          # detached signature over manifest (device verifies before install, $9.4)
```

Contract invariants (both halves depend on these): - order is a **global monotonic** sequence → assembly/playback is "sort by (chapter, order), concatenate." Nothing downstream parses text structure. - Audio filename **is** hash(text+voice+emotion+engine_version) → editing one line invalidates exactly one clip; the device caches by the same key (no divergence). - speaker_id is a foreign key into the character registry, or reserved NARRATOR. - emotion uses the fixed **7-state + intensity** vocabulary (replaces Hermes's free-text tone). - contract_version in the manifest gates compatibility; Hermes refuses a bundle it can't parse.

4. Provisioning — bundle → device

1. Studio publishes a signed bundle (release artifact / OTA channel / sideload).

2. Device **verifies the signature** over `manifest.json` before install (Part I §9.4 "signed book/content bundles"; §9.3 privacy — no child data leaves device; the bundle flow is one-way to the device).
3. Bundle lands under `/var/lib/hermes/books/<book_id>/` (encrypted at rest, Part I §8).
4. Device indexes it (§5) — no network needed thereafter; **full offline playback** (NFR-7).

5. PART B — Data → Hermes ingestion (mapping the contract onto the device model)

Hermes already has the receiving structures; ingestion is a **map**, not new subsystems.

Bundle element	→ Hermes structure	Notes
<code>utterances.jsonl</code> (ordered)	<code>story_agent indexed Segment list (segments_[] , pos_ pointer)</code>	today hand-authored as <code>[Speaker\ tone] text</code> in <code>data/stories/*.md</code> ; the bundle replaces that authoring with generated data. <code>order</code> → segment index.
<code>utterance.speaker_id</code> + <code>emotion</code>	<code>Segment.speaker</code> + <code>Segment.tone</code>	<code>tone</code> upgrades from free-text to the 7-state+intensity vocab (§6 alignment).
<code>characters.okf/</code>	OKF Character docs (curated knowledge, Part I §17.1–17.2)	casting + persona the connector reads to pick a voice.
<code>casting.json</code> + <code>voice_profiles.json</code>	voice selection in <code>llm_connector</code> for <code>PLAY_SEGMENT</code>	maps <code>speaker_id</code> → <code>voice_profile</code> so the character sounds identical across the book.
<code>audio/<hash>.opus</code>	Hermes clip cache (Part I §16.5 "resolve idx → cached clip")	filename is the cache key; <code>llm_connector</code> 's <code>PLAY_SEGMENT</code> handler resolves <code>segment_idx</code> → <code>utterance</code> → <code>audio hash</code> → <code>clip</code> .
<code>manifest.chapters</code>	<code>story_agent</code> chapter/position model + <code>STORY_DONE</code> boundaries	pause profile drives inter-segment gaps at assembly-parity.

Net: the bundle turns Hermes's 🛑 "book pipeline + cache" stub into a populated data set. The device code that consumes it — `story_agent` loop, `PLAY_SEGMENT`, the clip cache — is exactly the path in Part I §16.5, which is ⚠️ framework today (the `PLAY_SEGMENT` handler in `llm_connector` is the missing piece, Part I §21).

6. Data → Playback (the two audio sources at runtime)

At play time, Hermes narration is **cached clips**; interaction answers are **live TTS**. Both converge on the same PipeWire → speaker path.

Narration (cached, from the bundle) — the common case, Part I §16.5:

```
supervisor _Story::cmd::START → story_agent.play(0)
play(idx): StorySegmentRef{idx} - _Llm::cmd::PLAY_SEGMENT(0x504) → llm_connector:
  resolve idx → utterance → audio/<hash>.opus (clip cache HIT) → PipeWire → speaker
  _Llm::evt::TTS_STREAM_END(0x588) → story_agent.play(++pos_) ... until STORY_DONE(0x881)
```

Lowest latency, fully offline, no synthesis on device — the studio already paid the TTS cost once.

Interaction (live, NOT in the bundle) — Part I §16.6, UC-2/UC-3:

```
VTs wake / VAD barge-in → supervisor FSM (IDLE→CAPTURE→THINK→SPEAK)
STT → recall (mem0/OKF) → route() local|cloud LLM → GUARDRAIL GATE → live TTS → speaker
(voice/emotion from the same casting map so an answered character still sounds like itself)
resume narration at story_agent pos_
```

Real-time, single-utterance, on-device. This is the **device-live half** the studio can't pre-render (the child's questions aren't known ahead of time).

Division of labor: narration content = **Studio**; conversation, memory, safety, timing = **Hermes**. The casting map is shared so a live answer from "Sherlock" uses Sherlock's voice.

6.1 UC-8 — Conversing with a story character (role-load design)

The product moment this system exists for: the listener talks to the character, and the character answers in persona, in voice, knowing the story so far — and nothing beyond it.

Actors: listener (child) · `story_agent` · memory subsystem (mem0/OKF) · `llm_connector` · TTS (per DR-1). **Preconditions:** a bundle with Stage-® dossiers installed; reading position known (`BookFact.progress`); guardrails present (`brain/guardrails.md`).

Main flow (elaborates memory doc §4.1 with the character-knowledge inputs):

```
1 barge-in/wake → supervisor FSM → STT: "Genie, why were you stuck in the lamp?"
2 addressee resolution: "Genie" → characters.okf/ch_<genie>.md (aliases[] match)
3 ROLE-LOAD assembles the system prompt from FOUR read-only inputs:
  a. guardrails (brain/guardrails.md)      - safety, stay-in-role, deflection rules
  b. character dossier (bundle, Stage ⑥)    - persona, style, catchphrases
  c. dossier knowledge WHERE max_order ≤ pos_ - spoiler-scoped story facts
  d. recalled listener memory (mem0/OKF)   - "favorite character", prior Q&A
4 route() local|cloud LLM → in-character reply → GUARDRAIL GATE (post-check)
5 TTS with the character's voice_profile_id (same casting map as narration - continuity)
6 resume narration at story_agent pos_; capture the exchange → memory consolidation
```

Alternate flows: A1 addressee ambiguous → narrator voice asks "Who do you want to ask?"; A2 question outside scoped knowledge → in-persona deflection ("You'll find out soon..." — a guardrail template, not an LLM improvisation); A3 offline → local LLM + local TTS tier (DR-1), same role-load, reduced quality; A4 guardrail post-check fails → narrator-voice safe fallback.

Why the knowledge lives in the bundle, not the device: dossiers are per-book, authored once, reviewable/signable like every other bundle artifact, and identical across the fleet. The device contributes only what it uniquely knows: listener memory, reading position, and safety state. The runtime is **model-free about the story** — it never invents canon; it retrieves it.

6.2 DR-1 — Where TTS runs (decision record)

Decision (2026-07-12): hybrid, split by content mutability.

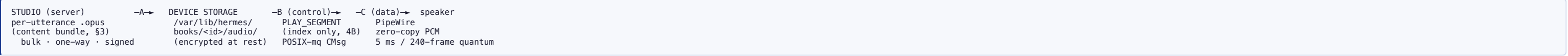
Content	TTS placement	Rationale
Narration + book dialogue (static)	Cloud, pre-rendered at build (Stage ⑥)	quality ceiling (engine class the RK3588 can't run); audition/fix before ship; <code>audio_key</code> caching makes it incremental; playback = Opus decode, ~zero CPU; fully offline (NFR-7)
Interactive answers, online	Cloud, streamed (§7.D path)	same engine as pre-render ⇒ perfect voice continuity; best prosody for open-ended text
Interactive answers, offline	On-device light TTS (Piper-class realtime on A76; NPU offload candidates)	keeps interaction alive without network; flatter prosody accepted as the documented degradation

Rejected: (i) all-realtime on device — recomputes identical audio every read-through, caps quality at embedded-class engines, competes with the LLM for compute, and forfeits the review gate; (ii) all-pre-rendered — cannot exist: interactive replies aren't known at build time.

Consequence — voice continuity contract: a character must sound the same from all three sources. Mitigation: pre-render and online streaming share one cloud engine; for the offline tier, Stage ⑤ casting gains a **local_voice** field per profile (closest local-engine voice, chosen deliberately at build time, not improvised on-device). This resolves open question (e) of §11. Schema addition is backward-compatible (`contract_version` minor bump).

7. Audio transport — how bytes reach the speaker

Audio is **never streamed to the device as a real-time feed**. Narration is pre-rendered *files*, bulk-provisioned once, then played entirely on-device. Three transports are kept **deliberately separate**, and audio rides exactly one of them at a time.



7.A Studio → device — provisioning (bulk file transfer, not streaming)

Rendered clips leave the Studio as **files in a signed bundle**, not a stream: - one file per utterance, named by its content hash — `audio/<hash>.opus` (`audio_key(text+voice+emotion+engine_version)` , §3). - the whole bundle is delivered **once, ahead of time** (OTA channel or sideload), **one-way** to the device, signature-verified before install (Part I §9.4), landed encrypted under `/var/lib/hermes/books/<id>/` . - "packetization" here is just the file-transfer/OS layer (TCP/USB); architecturally it is a **bulk artifact**, not audio frames. After it lands, playback needs **no network** (NFR-7). - **Status:** 🛑 — OTA/bundle transport is open decision (a) in §11.

7.B On-device control plane — indices, never audio

The **MsgBus (POSIX-mq)** carries only tiny POD; this is the load-bearing rule: - `CMsg` = **20-byte header + ≤256-byte inline body** (`CMsg.hpp`). Bulk audio is explicitly forbidden inline — "*bulk audio travels via PrerollRing/PipeWire, not here.*" - to play a clip, `story_agent` sends `PLAY_SEGMENT` with `StorySegmentRef { int32 segment_idx }` — **4 bytes**. The index is the entire "packet"; `llm_connector` resolves `idx` → utterance → `audio/<hash>.opus` . - **Status:** ✅ **contract** / ⚠️ the resolving `PLAY_SEGMENT` handler is unbuilt (Part I §21).

7.C On-device data plane — PipeWire zero-copy (where PCM actually moves)

PCM travels the **PipeWire graph in shared memory**, never the bus: - `llm_connector` (a PipeWire client) decodes the Opus clip → PCM and pushes it over a **PipeWire link** → `hermes.abox` → `out_0` → speaker. - transport unit = the **5 ms quantum (240 frames @ 48 kHz, 32-bit float)** — zero-copy SHM, no malloc, no locks (Part I §13.1). That quantum is the closest thing to "packetization," and it is local RAM, not a wire. - inter-process audio handoff (e.g. VTS wake pre-roll → abox) uses a **shared-memory ring** (`PrerollRing`), again not the mq. - **Status:** engine + quantum + abox loopback-tested (SVVR SYS-04); $\triangle$ the clip-decode-into- graph path is part of the unbuilt `PLAY_SEGMENT` handler.

7.D The one networked-audio exception — interactive answers

A question that isn't pre-rendered (barge-in, UC-2/3) can be answered by **cloud TTS** — the only streamed audio in the system: - it rides the `llm_connector` 's own **TCP socket** (chunked/streamed), surfaced on the bus only as small `TTS_CHUNK` **notifications** (events, not audio); the PCM is injected into PipeWire locally. - **PipeWire itself never reaches the network** — only the connector's socket does (Part II §16.1 / D7). **Status:** stub.

7.E Summary — what is packetized where

Path	Unit	Medium	Contains audio?	Status
Studio → device	whole <code>.opus</code> file, signed bundle	OTA / sideload (bulk)	yes (as files)	
Control plane	<code>CMsg</code> 20B + $\leq$ 256B; <code>PLAY_SEGMENT(idx)</code> 4B	POSIX-mq	no — index only	/ $\triangle$
Data plane (device)	5 ms / 240-frame float buffer	PipeWire zero-copy SHM	yes (PCM)	/ $\triangle$
Cloud interactive TTS	streamed chunks	connector TCP socket	yes	

Net: narration = pre-rendered files provisioned in bulk, then decoded and played through the local PipeWire quantum; the control bus only ever moves a segment index; the network is touched only for optional cloud answers, and never by PipeWire. **Open decision (d), §11:** clip codec — Opus (small bundle/OTA, costs an A55 decode per segment) vs. raw PCM (zero-decode, ~10× storage) vs. FLAC (lossless middle).

8. Build-path separation & repo structure

	Studio (Part A)	Hermes device (Part B)
Runtime	offline batch, server/GPU	hard-RT embedded, RK3588
Stack	Python, Celery/Redis, spaCy/Tesseract/ffmpeg, GPU TTS	C11 (RT) + C++17 (control), CMake, PipeWire/MsgBus
Build	its own Python env / container; excluded from the CMake device build	aarch64 cross-compile (<code>cmake/rk3588.toolchain.cmake</code>)
Cadence	once per book	continuous, per-utterance
Tested by	attribution accuracy, auto-resolve-above- τ %, render cost	ctest + PipeWire loopback (SVVR)

Recommended layout (monorepo, separate path — consistent with the existing `services/memory/server.py` Python sidecar precedent):

```
Hermes/
  app/ common/ ...      # device firmware (C/C++, aarch64)      ← Part B
  services/memory/...   # existing Python sidecar
  studio/               # NEW: Book-to-Voice Studio (Python) ← Part A   [excluded from CMake]
    ingest/ nlp/ attribute/ emotion/ casting/ tts/ assembly/
    contract/           # bundle schema + validator (the §3 seam) — the ONE shared spec
    docs/               # this doc, ARCHITECTURE.md, ...
```

A separate repo is equally valid; the hard rule either way: **Studio never enters the aarch64 build, and the bundle contract (`studio/contract/`) is the single shared artifact.**

9. Phased build order (aligning Studio sprints with Hermes phases)

Milestone	Studio (Part A)	Hermes (Part B)	Proves	Status
M0	ingest + data model + narrator-only render (one voice)	ingest bundle → <code>story_agent</code> plays cached clips end-to-end (finish §16.5: <code>PLAY_SEGMENT</code> handler + clip cache)	the whole book→playback path with one voice	Studio: data half (ingest incl. scanned-PDF/OCR + bundle, §2.1–2.2); render half = Stage ③ . Device: $\triangle$
M1	Stage ③ explicit-tag attribution + review UI	—	de-risk the crux; the review loop is the product	tiers 1–5 + τ review queue (CLI print; web UI)
M2	coreference + alias-merge + registry + casting	map <code>characters.okf</code> + casting → OKF + voice profiles; per-character voices	characters get distinct, stable voices	Studio: rules registry + seeded casting; coref/spaCy $\triangle$. Device:

Milestone	Studio (Part A)	Hermes (Part B)	Proves	Status
M2.5	Stage ⑧ character dossiers (§2.3) + local_voice casting field (DR-1)	role-load consumes dossiers + spoiler scoping	a character you can <i>talk to</i> (UC-8)	Studio: ✅ scenes+dossiers (contract v0.2); local_voice ❌. Device role-load: ❌
M3	emotion tagging + expressive TTS + mastering	tone→7-state in story_agent ; expressive clip playback	polish once correctness is solid	Studio: cue-tier ✅; expressive TTS ❌
M4	local-GPU TTS + content-hash cache + scale-out	clip cache keyed by the bundle hash	collapse marginal cost; cheap re-renders	❌ (cache key + per-page idempotent shaping already in place)
M5	—	interaction layer: VTS wake, VAD barge-in, live LLM+TTS answer, memory, guardrail gate	the <i>interactive</i> audiobook (Part I §16.6, UC-2/3, UC-8 §6.1)	❌ (memory sidecar △)

M0–M4 build the **narration** product across both halves; M5 adds Hermes's **interaction** on top. This matches the PDF's "boring end-to-end path first, then the crux, then polish" and Hermes's own P0–P5 roadmap (Part I §23).

10. What to align now (before the Studio exists)

Cheap moves that make the two halves contract-compatible from day one: 1. **Adopt the utterance-stream schema** as Hermes's story format — evolve story_agent 's [Speaker|tone] line parser toward utterances.jsonl (or generate the line format *from* it), so hand-authored test scripts and generated bundles share one shape. 2. **Adopt the content-hash cache key** hash(text+voice+emotion+engine_version) for Hermes's clip cache now, so device and studio never diverge on what a clip is named. 3. **Fix the 7-state emotion vocabulary** in story_agent /casting (replace free-text tone), matching Stage ④ and the connector's expressive-TTS conditioning. 4. **Define studio/contract/** (schema + validator) as the shared source of truth, and have both Part I §16.5 playback and any Studio prototype validate against it. 5. **Upgrade ARCHITECTURE.md §17.7** from the 3-line stub to "see this doc," and mark UC-7 as *designed (offline studio), integration-specified* rather than a bare ❌ .

11. Status & open questions

- **Today (2026-07-12):** Part A **data + knowledge halves built and validated** — studio/ implements stages ①–⑤+⑦+⑧ (rules tiers throughout; LLM tier for ⑧ persona authoring) with scanned-PDF/OCR ingest, proven on a real image-only picture-book scan (§2.1–2.3: contract v0.2 valid, deterministic, 32 tests). The bundle now carries everything UC-8 role-load consumes (dossiers + spoiler-gated scenes). Part A render half (Stage ⑥ TTS) is ❌ — the next Studio milestone, per DR-1 cloud-side. Part B narration path remains △ framework (§16.5; PLAY_SEGMENT handler ❌); interaction layer (M5/UC-8) ❌ with the memory sidecar △ in place.
- **Resolved since v0.1:** (e) live-answer voice provisioning → **DR-1** (§6.2): shared cloud engine online + build-time local_voice mapping for the offline tier.
- **Open:** (a) bundle transport/OTA vs. sideload; (b) whether Hermes ever runs a *local* Studio for user-supplied books, or Studio is always upstream; (c) exact OKF Character ↔ registry field mapping (Stage ⑧ will force this); (d) clip audio codec (opus vs. pcm) vs. device decode budget; (f) Stage ⑥ engine selection (XTTS-class self-hosted vs. commercial API — licensing/cost trade, §"TTS placement" discussion) ; (g) dossier review tooling (extend the τ review UI to cards).

This is the design behind ARCHITECTURE.md UC-7 / §17.7. Offline-half source: *Book-to-Voice Architecture.pdf* . Device-half source of truth: Part I §16.5–16.6, story_agent , llm_connector , StoryMsg.hpp . Uncommitted pending review.